

ESP32 7 — Two-Way Communication with the Server

Tutorial Group: ESP32

Goal

In this tutorial, the ESP32 will request information from the Python server and use the server's response. This completes the basic client/server communication cycle and demonstrates that the ESP32 can both send information to and receive information from a computer.

The Python server from *ESP32 6 — Sending Structured Data with JSON* is reused and extended. The ESP32 will make an HTTP GET request, and the server will return JSON.

What You Need

- ESP32-WROOM-32
- USB cable
- Computer with the Arduino IDE and Python 3
- Wi-Fi network
- ArduinoJson library installed

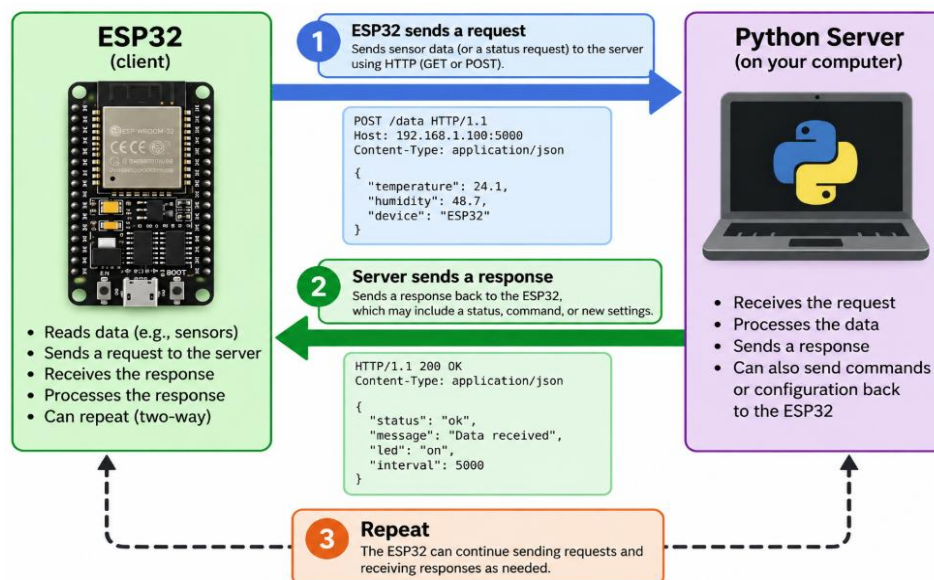
Part 1 — Two-Way Communication

In previous tutorials, the ESP32 sent information to the server with POST. Here, the ESP32 asks the server for information with GET.

This is two-way communication at the application level: the ESP32 sends a request and the server sends a response. It is not a persistent connection.

POST: ESP32 → Server

GET: ESP32 ← Server



Part 2 — Update the Python Server

Add this `do_GET()` method to your Python server:

```
def do_GET(self):
    response = {
```

```

        "message": "Hello from the server",
        "interval": 5000
    }

    data = json.dumps(response).encode("utf-8")

    self.send_response(200)
    self.send_header("Content-Type", "application/json")
    self.send_header("Content-Length", str(len(data)))
    self.end_headers()

    self.wfile.write(data)

```

The server returns JSON containing a message and an interval. The complete server should retain the POST handling from *ESP32 6 — Sending Structured Data with JSON* and add this GET handler.

Part 3 — Make an HTTP GET Request

On the ESP32, create the HTTP client and request the server:

```

HTTPClient http;

http.begin(serverURL);

int responseCode = http.GET();

Serial.print("HTTP Response: ");
Serial.println(responseCode);

```

Part 4 — Read the Server Response

If the request succeeds, obtain the response body:

```

String response = http.getString();

Serial.println(response);

```

The response will contain JSON similar to:

```

{
  "message": "Hello from the server",
  "interval": 5000
}

```

Part 5 — Parse the Response

Use `ArduinoJson` to interpret the response:

```

JsonDocument doc;

deserializeJson(doc, response);

const char* message = doc["message"];
int interval = doc["interval"];

```

```
Serial.println(message);  
Serial.println(interval);
```

Part 6 — Use Information from the Server

The important idea is that the ESP32 is no longer simply printing information received from the server. It can use the information in its own program.

For example, instead of using a fixed interval:

```
const unsigned long postInterval = 10000;
```

you could use a value supplied by the server. This creates a simple example of a network-controlled embedded system.

Part 7 — Complete ESP32 Sketch

```
#include <WiFi.h>  
#include <HTTPClient.h>  
#include <ArduinoJson.h>  
  
const char* ssid = "YourNetworkName";  
const char* password = "YourPassword";  
const char* serverURL = "http://192.168.1.50:5000/";  
  
void setup()  
{  
    Serial.begin(115200);  
  
    WiFi.begin(ssid, password);  
  
    while (WiFi.status() != WL_CONNECTED)  
    {  
        Serial.print(".");  
        delay(500);  
    }  
  
    Serial.println(" Wi-Fi connected");  
  
    HTTPClient http;  
  
    http.begin(serverURL);  
  
    int responseCode = http.GET();  
  
    Serial.print("HTTP Response: ");  
    Serial.println(responseCode);  
  
    if (responseCode == 200)  
    {  
        String response = http.getString();  
  
        Serial.println("Server response:");  
        Serial.println(response);  
    }  
}
```

```

    JsonDocument doc;

    DeserializationError error = deserializeJson(doc, response);

    if (!error)
    {
        const char* message = doc["message"];
        int interval = doc["interval"];

        Serial.print("Message: ");
        Serial.println(message);

        Serial.print("Interval: ");
        Serial.println(interval);
    }
    else
    {
        Serial.println("JSON parsing failed");
    }
}

http.end();
}

void loop()
{
    // This first tutorial performs the request once in setup().
}

```

Part 8 — Experiment

- Change the message returned by the server.
- Change the interval returned by the server.
- Add another field to the JSON response.
- Modify the ESP32 so that it uses the returned interval when scheduling another operation with `millis()`.
- Modify the server so that the response changes based on information received in a POST.

What You Should Understand

- POST sends information from the ESP32 to the server.
- GET allows the ESP32 to request information from the server.
- The server sends an HTTP response to the ESP32.
- JSON can be used in both directions.
- The ESP32 can use information received from a server to affect its own operation.
- A networked embedded system can communicate in both directions.

Before Moving On

You should be able to answer **yes** to all of the following:

- ☐ I can make an HTTP GET request.
- ☐ I can read the response from a server.
- ☐ I can parse JSON received by the ESP32.
- ☐ I understand the difference between POST and GET.

- ☐ I understand two-way client/server communication.
- ☐ I can use server-provided information in an ESP32 program.